

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчет по лабораторной работе №1
по дисциплине
«Объектно-ориентированное программирование»

Выполнил студент гр. ИВТб-2301-05-00	_____ /Макаров С.А./
Руководитель преподаватель	_____ /Шмакова Н.А./

Киров 2026

Цель работы

Цель работы: изучить и освоить принципы объектно-ориентированного программирования, включая инкапсуляцию, наследование и полиморфизм, путем создания иерархии классов в выбранной предметной области.

Задание

1. Создать иерархию классов состоящую не менее чем из одного родительского и двух дочерних классов.
2. В каждом классе определить не менее двух член данных. не менее двух собственных, а для дочерних не менее двух унаследованных и двух перекрытых член функций.
3. Разработать приложение демонстрирующее принципы инкапсуляции, наследования и полиморфизма.

Решение

Предметная область описывает управление ассортиментом блюд в заведении общественного питания. Система позволяет хранить информацию о различных блюдах, учитывать их специфические характеристики, рассчитывать итоговую цену для клиента в зависимости от этих характеристик и выводить информацию о блюде в удобном виде.

Программа содержит классы «Dish», представляющий базовый класс блюда, «Pizza», наследованный от «Dish» и представляющий класс пиццы, «Salad» наследованный от «Dish» и представляющий класс салата.

Базовый класс «Dish» содержит:

- Поля:
 - `private std::string name` – приватное поле, означающее название блюда, строковый тип данных;
 - `private int weight` – приватное поле, означающее вес блюда, целочисленный тип данных;

- `private double price` – приватное поле, цена блюда, формат с плавающей запятой;
- Конструкторы:
 - `public Dish()` – публичный конструктор, задает значения полей по умолчанию;
 - `public Dish(std::string name)` – публичный конструктор, принимает название блюда в качестве параметра;
 - `public Dish(std::string name, int weight, double price)` – публичный конструктор, принимает в качестве параметров название, вес и цену блюда;
- Геттеры/сеттеры:
 - `public std::string GetName() const` – геттер, возвращает название блюда;
 - `public int GetWeight() const` – геттер, возвращает вес блюда;
 - `public double GetPrice() const` – геттер, возвращает цену блюда;
 - `public void SetName(std::string name)` – сеттер, задает значение названию блюда, принимает в качестве параметра название блюда;
 - `public void SetWeight(int weight)` – сеттер, задает значение весу блюда, принимает в качестве параметра вес блюда;
 - `public void SetPrice(double price)` – сеттер, задает значение цене блюда, принимает в качестве параметра цену блюда;
- Абстрактные методы:
 - `public virtual void DisplayInfo() = 0` – публичный виртуальный метод, отображающий информацию о блюде, не принимает параметров;
 - `public virtual double CalcFullPrice() = 0` – публичный виртуальный метод, возвращающий полную цену блюда.

Класс «Pizza», наследованный от класса «Dish» содержит:

- Собственные поля:
 - `private Dough dough = Dough::Thick` – приватное поле, означающее типа теста, по умолчанию `Thick`;
- Конструкторы:
 - `public Pizza() : Dish()` – публичный конструктор, задает значения полей по умолчанию;
 - `public Pizza(std::string name) : Dish(name)` – публичный конструктор, принимает название пиццы в качестве параметра;
 - `public Pizza(std::string name, int weight, double price) : Dish(name, weight, price)` – публичный конструктор, принимает в качестве параметров название, вес и цену пиццы;
- Геттеры/сеттеры:
 - `public Dough GetDough() const` – геттер, возвращает тип теста;
 - `public void SetDough(Dough dough)` – сеттер, задает значение типу теста, принимает в качестве параметра `enum Dough (Thick, Thin)`;
- Переопределенные методы:
 - `public void DisplayInfo() override` – публичный метод, отображает информацию о пицце (название, тип теста, вес, цена), не принимает параметров, выводит информацию в следующем виде:
`Pizza: Margarita, thick dough, 450g, $18.00;`
 - `public double CalcFullPrice() override` – публичный метод, возвращает полную цену в зависимости от типа теста (`Thick: price * 1.5`, `Thin: price * 1.3`), не принимает параметров;
- Собственные методы:
 - `public void ChangeDough()` – публичный метод, меняет тип теста, не принимает параметров;

- `public void CutInSlices()` – публичный метод, нарезает пиццу на куски, не принимает параметров, выводит информацию в следующем виде:

`Cutting the pizza into slices...`;

Класс «Salad», наследованный от класса «Dish» содержит:

- Собственные поля:
 - `private Dressing dressing` – приватное поле, означающее приватное поле, о тип заправки, по умолчанию `OliveOil`;
- Конструкторы:
 - `public Salad() : Dish()` – публичный конструктор, задает значения полей по умолчанию;
 - `public Salad(std::string name) : Dish(name)` – публичный конструктор, принимает название салата в качестве параметра;
 - `public Salad(std::string name, int weight, double price) : Dish(name, weight, price)` – публичный конструктор, принимает в качестве параметров название, вес и цену салата;
- Геттеры/сеттеры:
 - `public Dressing GetDressing() const` – геттер, возвращает тип заправки;
 - `public SetDressing(Dressing dressing)` – сеттер, задает значение типу заправки, принимает в качестве параметра `enum Dressing (OliveOil, Mayonnaise)`;
- Переопределенные методы:
 - `public void DisplayInfo() override` – публичный метод, отображает информацию о салате (название, тип заправки, вес, цена), не принимает параметров, информацию в следующем виде:
`Salad: Caesar, OliveOil dressing, 250g, $12.60;`

- `public double CalcFullPrice()` override – публичный метод, возвращает полную цену в зависимости от типа заправки (OliveOil: `price * 1.4`, Mayonnaise: `price * 1.3`), не принимает параметров
- Собственные методы:
 - `public void ChangeDressing()` – публичный метод, меняет тип заправки, не принимает параметров;
 - `public void TossWithDressing()` – публичный метод, перемешивает салат с заправкой, не принимает параметров, выводит информацию в следующем виде:

Tossing the salad with OliveOil...;

Диаграмма классов представлена на рисунке 1.

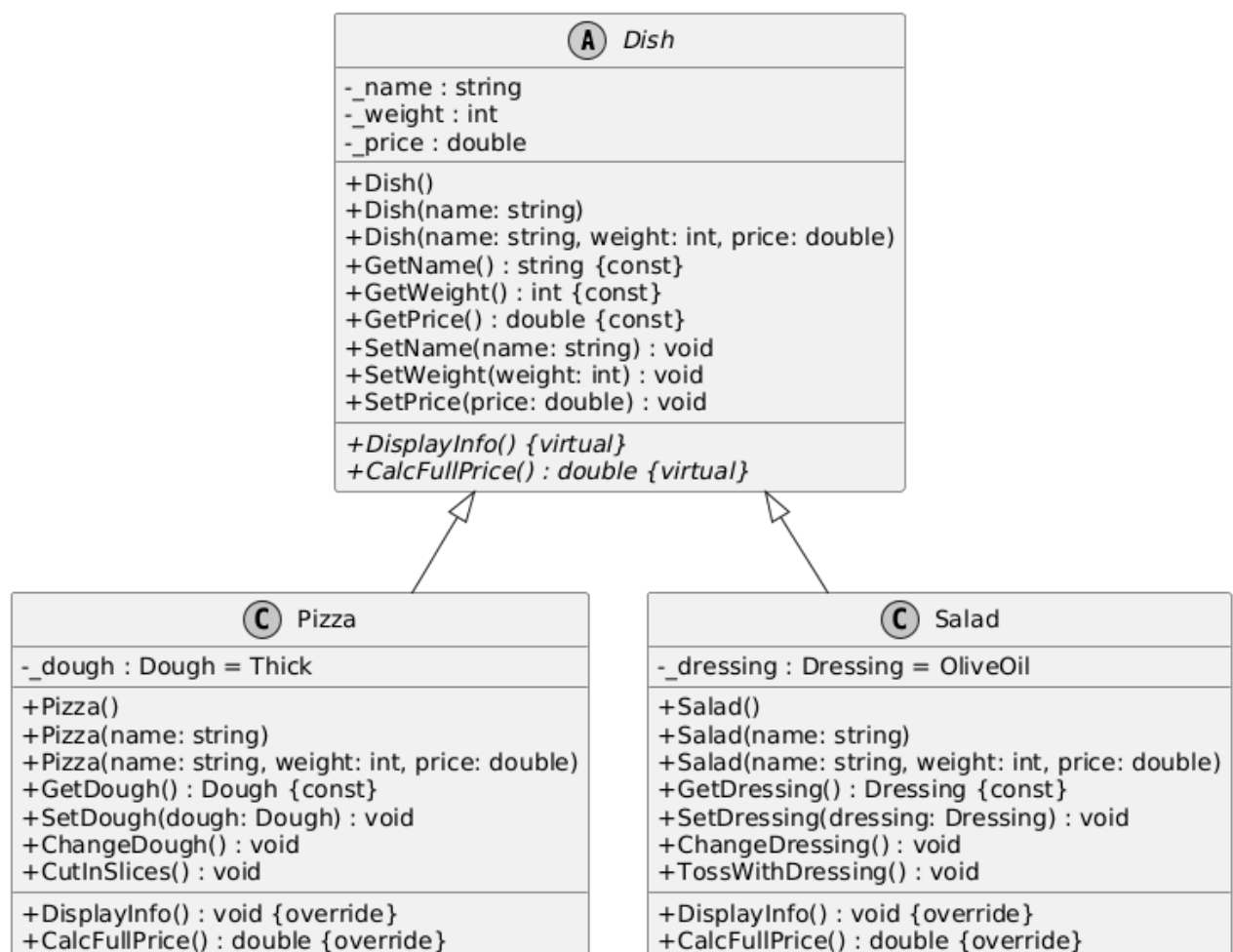


Рисунок 1 – Диаграмма классов

Вывод

В ходе выполнения лабораторной работы была разработана и реализована иерархия классов, моделирующая управление ассортиментом блюд в заведении общественного питания. Изучены основные принципы объектно-ориентированного программирования: инкапсуляция, наследование, полиморфизм. Разработано приложение, демонстрирующее применение данных принципов.